Should the Fusion Weight Change with the Question? A Study of Continually Learned Fusion Weights for Keyframe Search

Jianxiang He
CV805 Course Project
jianxiang.he971@gmail.com

Abstract

To answer a question about a long video, a vision-language model can only look at a few frames, so we first have to pick the right ones. VSI picks frames with two signals: what objects appear in each frame, and how well the subtitle at that moment matches the question. It adds the two scores with one fixed weight λ. We ask whether this weight should change with the type of question, and whether a small model can learn it when question types arrive one after another (continual learning). On LongVideoBench we split 753 questions into four question types and train a small λ predictor on them in sequence. We find three things. First, the best weight clearly depends on the question type: questions about subtitles prefer a large λ (mean 0.62) and questions about visual content prefer a small one (mean 0.49, $p = 1.6 \times 10^{-6}$). Second, the learned predictor beats the default $\lambda = 0.3$ by about 5 points of keyframe recall in all 9 runs, but it only ties a single λ tuned on the training set, because it outputs almost the same λ for every question type. Third, better keyframe recall barely improves question answering, except on the 38% of questions where λ actually changes the selected frames. Our results suggest that a well-tuned constant is the baseline any learned weight must beat, and that predicting the per-question differences is the real open problem.

1. Introduction

A long video can last tens of minutes, but a vision-language model (VLM) can only take a few frames as input. Which frames we give it often decides whether it answers correctly. Recent *keyframe search* methods therefore look for the frames that are most related to the question before calling the VLM [9, 12, 16].

VSI [5] is one such method. For every second of the video it computes two scores (Fig. 1). The *visual score* checks whether the objects mentioned in the question appear in the frame, using the open-vocabulary detector YOLO-World [4]. The *subtitle score* measures how similar the subtitle at that second is to the question, using sentence embeddings [11]. VSI then adds the two scores with a weight λ and keeps the 8 highest-scoring seconds:

$$S_\lambda(t) = (1-\lambda)\,\hat{s}_{\text{vis}}(t) + \lambda\,\hat{s}_{\text{sub}}(t), \tag{1}$$

where $\hat{s}$ means the score rescaled to $[0, 1]$. $\lambda = 0$ uses only the visual score and $\lambda = 1$ uses only the subtitles. VSI uses $\lambda = 0.3$ for every question.

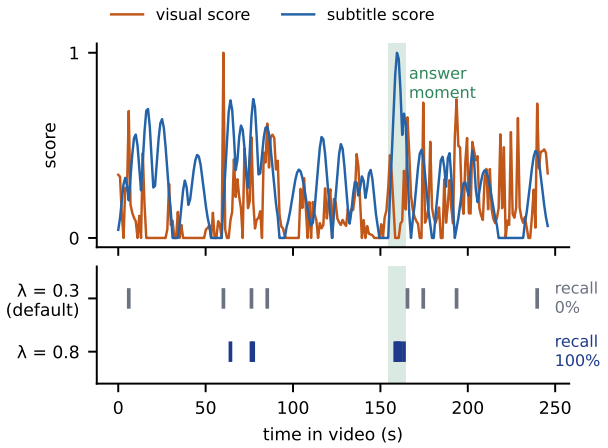


Figure 1. **Why the weight matters.** One real question from LongVideoBench ("…when the spoon places the meat onto the mashed potatoes and the subtitle mentions 'Just pour the…'"). Top: the two scores over time; the green band is the annotated moment. Bottom: the 8 seconds selected with the default $\lambda = 0.3$ all miss that moment, while $\lambda = 0.8$ puts most of them on it.

A single weight for all questions seems too simple. A question about what is *said* should rely more on subtitles; a question about what is *shown* should rely more on the picture. Figure 1 shows a real case where changing λ from 0.3 to 0.8 turns a complete miss into a hit. A natural idea is to train a small model that looks at each question and outputs its own λ. In practice, new types of questions keep appearing, so this model should be able to learn new ques-

tion types one after another without forgetting the old ones. This setting is called *continual learning* [7, 13].

This report studies four questions:

- **Q1.** Does the best λ really depend on the question type? How much can we gain, and where does the gain come from?
- **Q2.** Can a λ predictor trained on question types in sequence learn it? Does it forget?
- **Q3.** If the gain is small, what is holding it back?
- **Q4.** Do better selected frames lead to better answers?

Our contributions are: **(1)** We turn VSI's fusion weight into a continual learning task on LongVideoBench [14], with four question types, three task orders and three random seeds, and with fair baselines, including a constant tuned on the training set. **(2)** We show that the best weight depends strongly on the question type, and we split the possible gain into three parts: a better global constant, one constant per question type, and one weight per question (Fig. 4). **(3)** We find that the learned predictor only matches a tuned constant, because it gives nearly the same λ to all question types (Fig. 6). **(4)** We test the selected frames with a real VLM and find that a better λ only helps the minority of questions where λ changes the frames. We also describe four implementation mistakes that each reversed our conclusions during the project (Sec. 7).

2. Related Work

Keyframe selection for long videos. Sampling frames at even intervals can miss short but important moments. T* [16] treats frame selection as a search over time guided by an object detector. AKS [12] and BOLT [9] select frames by their relevance to the question without training the VLM. VSI [5] adds a subtitle signal to T* and mixes the two signals with a fixed weight. These methods usually keep their mixing or sampling settings fixed for all questions; we study whether the mixing weight itself should adapt.

Adaptive fusion of modalities. Dynamic fusion methods change how much each modality contributes depending on the input [15]. In VSI both branches are fixed, off-the-shelf models and the only thing to learn is one number, λ. This makes it easy to measure exactly how much learning the weight is worth.

Continual learning. When a model is trained on tasks one after another, it tends to lose what it learned on earlier tasks; this is called forgetting. Common remedies keep important weights close to their old values (EWC [6]), make the new model imitate the old model's outputs (LwF [8]), or replay a few stored examples from old tasks (ER [3]). Results are usually reported as the final average accuracy and the amount of forgetting [2, 10]. Most prior work studies classification with many classes; here the output is a single number and the training signal is sparse, which lets us look directly at what the model learns and forgets.

3. Setup

Data and question types. We use the validation set of LongVideoBench [14] and take one question per video, 753 questions in total. Using the official question categories, we group them into four tasks: T1 text perception (127 questions, *e.g.* "which subtitle appears when…"), T2 visual perception (237), T3 text relation (171), and T4 visual relation (218). Each task is split randomly into 70% training and 30% test questions. We use three random splits (seeds) and three task orders: default T1→T2→T3→T4, grouped T1→T3→T2→T4, and random T3→T1→T2→T4. Every method is therefore run $3 \times 3 = 9$ times. We compute the two VSI scores once with the official code and store them, so all training runs take minutes on a CPU.

How we measure frame selection. Each question comes with annotated moments (on average about two). *Recall@8* is the fraction of these moments that have at least one of the 8 selected seconds within ± 5 seconds. For each question, recall can be computed for any λ on a grid $\{0, 0.1, \ldots, 1\}$.

The λ predictor. The predictor is a small two-layer network (54 inputs, 128 hidden units, 11 outputs). Its 11 outputs are probabilities for the 11 grid values of λ, and the weight it uses is the probability-weighted average. Its 54 inputs are computed only from the two score curves, without any annotation: 10 basic statistics of the curves, plus 44 features that describe each curve's shape, whether the two curves agree, and how much the top-8 frames change when λ changes. Inputs are standardized with statistics from the first task only, so no information leaks from later tasks.

Training signal. For about 60% of the questions, recall does not change at all with λ, and for the rest it takes only a few values. Using "the best λ" of each question as a class label therefore gives almost no signal. Instead, we train the predictor to rank the annotated seconds high in the fused curve. With $\bar{\lambda}$ the predicted weight and $\tau = 0.1$:

$$\mathcal{L} = -\log \sum_{t \in \mathrm{GT}} \mathrm{softmax}\big(S_{\bar{\lambda}}/\tau\big)_t. \tag{2}$$

Each task is trained for 50 epochs with Adam (learning rate 10^{-3}, batch size 64).

Continual learning methods. *Naive* simply continues training on each new task. *EWC* [6] penalizes changes to weights that were important for old tasks. *LwF* [8] asks the

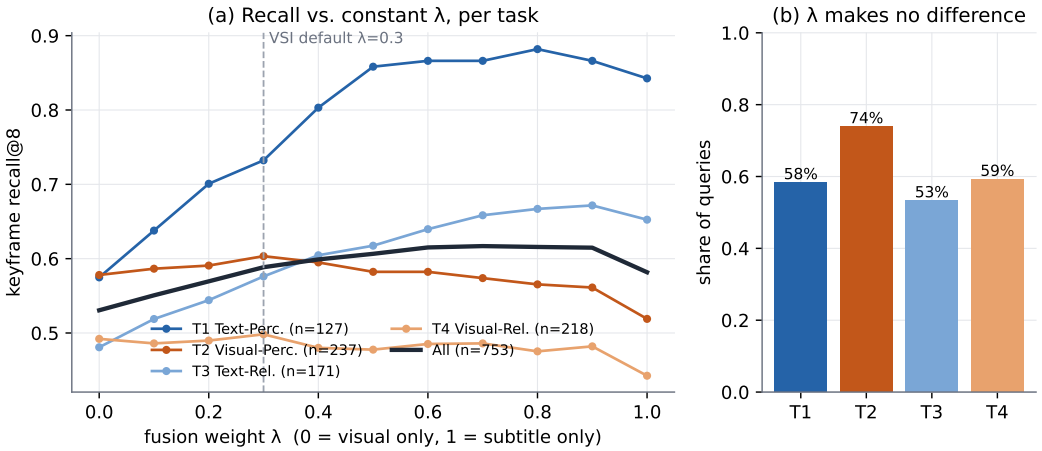


Figure 2. **The best weight depends on the question type.** (a) Recall@8 of every task as a function of a constant λ. Text tasks (blue) peak at $\lambda = 0.8$–0.9; visual tasks (orange) peak at $\lambda \leq 0.3$. The dashed line is the VSI default. (b) Share of questions whose recall does not change at all with λ.

model to keep its old outputs on new data. *ER* [3] replays up to 200 stored old questions. *Joint* retrains on all tasks seen so far; it is not a continual method but a reference upper bound. *kNN* stores all seen questions and copies the best λ of the 5 most similar ones.

Baselines. We compare against four constants and limits: the VSI default $\lambda = 0.3$; *one tuned* λ, the single value with the best average training recall (usually 0.7); *one* λ *per task*, tuned on each task's training questions (this needs to know the question type); and the *oracle*, which picks the best λ for each test question using its annotation, as an upper limit.

Metrics. Let $R_{i,j}$ be the test recall on task j after training on the i-th task, with $T = 4$ tasks. The final accuracy is ACC $= \frac{1}{T}\sum_j R_{T,j}$. Forgetting is FM $= \frac{1}{T-1}\sum_{j<T}\left(\max_{i<T} R_{i,j} - R_{T,j}\right)$, *i.e.* how far each old task has dropped from its best value; lower is better.

4. Q1: Should the Weight Depend on the Question?

Different tasks want different weights. Figure 2a shows recall for every constant λ. The two text tasks are best at $\lambda = 0.8$ to 0.9, while the two visual tasks are best at $\lambda = 0.3$ or lower. The difference is largest for T1: moving λ from 0.3 to 0.8 raises its recall from 0.73 to 0.88. Over all 753 questions, the best single value is $\lambda = 0.7$ (recall 0.617), about 3 points above the default 0.3 (0.588). So the default is too low for this dataset.

The same holds question by question. For the 284 questions where λ matters, we take the middle of each question's best-λ range. Its mean is 0.62 for text questions and 0.49 for visual questions (one-sided Mann-Whitney test, $p = 1.6 \times 10^{-6}$; Fig. 3).

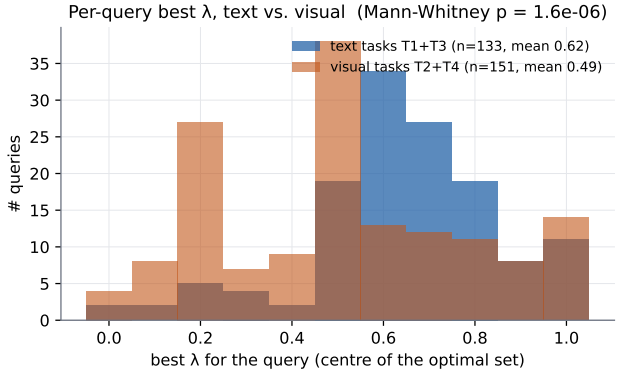


Figure 3. **Best weight per question.** For the 284 questions where λ matters, the best λ of text questions (blue) is clearly higher than that of visual questions (orange).

Where is the room for improvement? Figure 4 splits the gap between the default and the oracle into three steps. Tuning one global constant gains 4.5 points. Allowing one constant per task adds only 1.4 more. Choosing the best weight for every single question would add another 8.9. At the same time, 53% to 74% of the questions in each task do not react to λ at all (Fig. 2b). In short, the question type matters in a statistical sense, but most of the remaining gain lies in per-question differences, and only a minority of questions carry any signal about them.

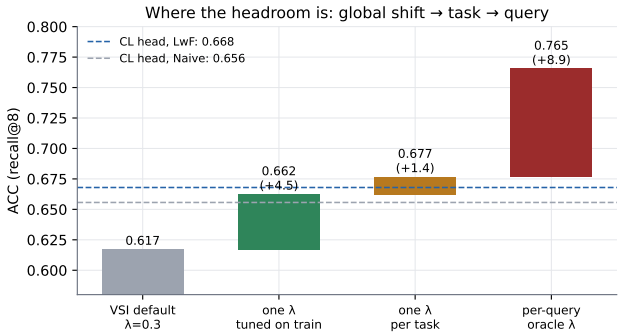


Figure 4. **Where the possible gain is.** Final accuracy (ACC) when going from the default weight to one tuned constant, to one constant per task, and to the best weight for every question. Dashed lines: two learned predictors. Mean of 9 runs.

Method	ACC ↑	FM ↓	vs. 0.3	vs. tuned
Default $\lambda = 0.3$ (VSI)	0.617	–	–	–
One tuned λ	0.662	–	+4.5	–
One λ per task†	0.677	–	+6.0	+1.4
Oracle (per question)‡	0.765	–	+14.8	+10.3
Naive	0.656	0.029	+3.9	−0.7 (3/9)
EWC	0.662	0.023	+4.5	−0.1 (4/9)
LwF	**0.668**	**0.015**	+5.1	+0.6 (7/9)
ER	0.660	0.021	+4.3	−0.3 (4/9)
Joint (reference)	**0.668**	**0.015**	+5.2	+0.6 (6/9)
kNN	0.645	0.005	+2.9	−1.7 (0/9)

Table 1. **Main results** (mean of 3 orders × 3 seeds). The last two columns give the ACC difference in points; in brackets, how many of the 9 runs beat the tuned constant. Standard deviations of ACC are 0.007–0.013. †Needs the question type. ‡Uses test annotations; an upper limit only.

5. Q2 and Q3: What Does Continual Learning Learn?

Main result. Table 1 summarizes the 9 runs. Every learned method beats the default $\lambda = 0.3$, by 3.9 to 5.2 points, in every single run (Fig. 5a). However, compared with one constant tuned on the training set (0.662), only LwF and Joint are slightly better (+0.6 points, in 7 and 6 of 9 runs). Naive, EWC and ER are about equal to it, and kNN is worse. No method reaches one constant per task (0.677), and all are about 10 points below the oracle.

Forgetting. LwF and Joint forget about half as much as Naive (FM 0.015 vs. 0.029), with EWC and ER in between. But this holds in only 6 or 7 of the 9 runs, and the spread across runs is as large as the difference. We can only say that the anti-forgetting methods tend to forget less. Forgetting is also small for every method; the next paragraph explains why.

Q3: the predictor learns one weight for everything. Figure 6 shows the average λ the predictor gives to each task after each training stage. The four lines almost overlap. The predictor does *not* give text tasks a larger weight and visual tasks a smaller one; it moves all questions to about 0.6, close to the tuned constant of 0.7. This explains Tab. 1: the predictor captures the "better global constant" step of Fig. 4, but not the "per task" or "per question" steps. It also explains why forgetting is small: since all tasks end up with similar weights, learning a new task does not overwrite much. We can still see forgetting in the figure: after training on T2 (visual perception), Naive drops all weights to 0.55 and moves back up after T3, while LwF changes much less.

Why can it not learn the differences? We see two reasons. First, the training signal is sparse: 62% of the questions do not react to λ, so almost all of the learning comes from the remaining third. Second, the inputs do not tell the predictor the question type; they only describe the score curves, which look similar across tasks. Adding the 768-dimensional embedding of the question text did not help either: it made test recall lower, which points to overfitting on this small dataset (appendix, Fig. 10).

6. Q4: Do Better Frames Give Better Answers?

Setup. We give the 8 frames chosen by each method, together with the subtitle shown at each frame, to Qwen3-VL-8B-Instruct [1] and ask it to answer the multiple-choice question. The learned predictors are tested only on the test questions of their own split (229 per seed), and are compared with the default weight on exactly the same questions.

Results. Over all questions, the learned weight and the default weight give the same accuracy (Tab. 3). Even the oracle weight adds only 1.8 points. The constant with the best recall, $\lambda = 0.7$, answers slightly *worse* than 0.3. So recall@8 is only a rough stand-in for answer quality.

Where the weight helps. Hitting the right moment does matter: at $\lambda = 0.3$, accuracy is 0.49 when the 8 frames hit none of the annotated moments and 0.75 when they hit all of them (Fig. 7a). We then split the questions into two groups (Fig. 7b). On the 469 questions where λ does not change the frames, all weight choices give the same answers, as expected. On the 284 questions where it does, the oracle weight answers 5 points more questions correctly than the default (0.673 vs. 0.623). The LwF predictor gains 1.5 points on its 266 such test questions, but the confidence interval $[-2.3, +5.6]$ still includes zero. In other words, the weight does matter for answering, but only for about 38%

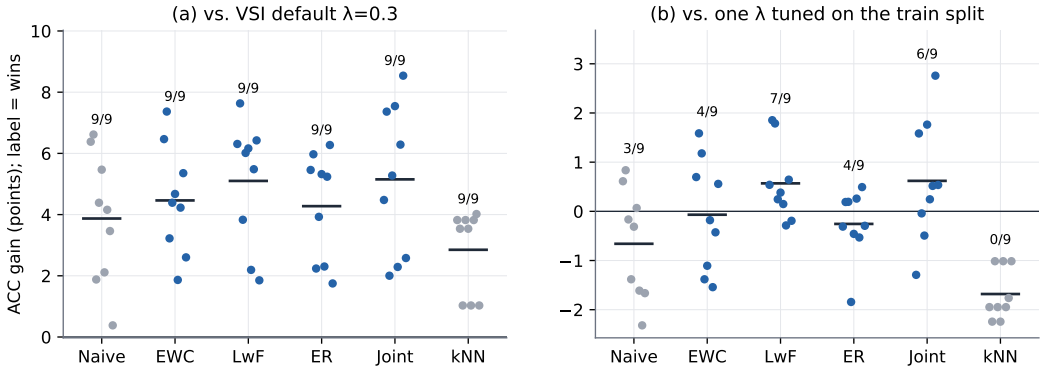


Figure 5. **Run-by-run comparison.** Each dot is one task order and seed; the bar is the mean; the number above is how many of 9 runs are better. (a) Against the default $\lambda = 0.3$: always better. (b) Against one λ tuned on the training set: about even.

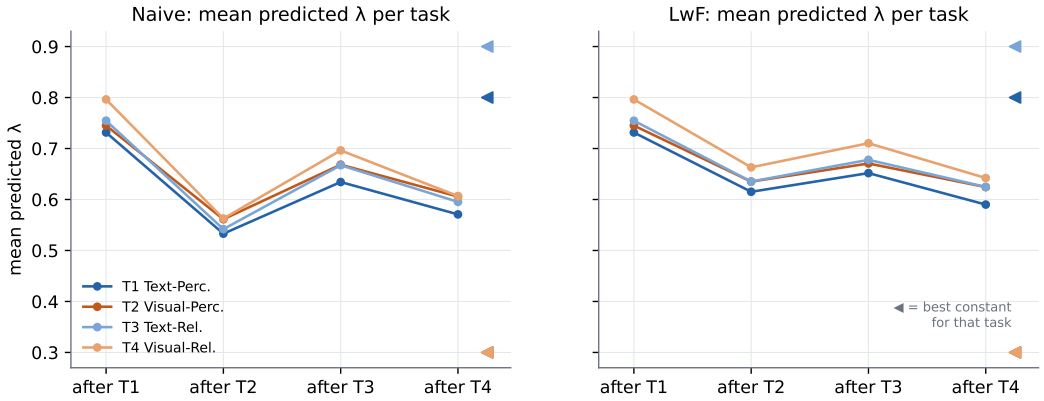


Figure 6. **The predictor gives every task almost the same weight.** Mean predicted λ on each task's test questions after training on each task (default order, mean of 3 seeds). Triangles on the right mark the best constant of each task. The four lines nearly overlap, far from the task-specific targets.

of the questions, and the current predictor captures only a small part of that.

7. Four Mistakes That Reversed Our Conclusions

During the project, four problems let every experiment run without any error message but produced wrong results (Tab. 2). Each of them changed our conclusion at some point. For example, before we fixed the last one, every learned weight looked 3 to 5 points *worse* than the default, which would have led us to say that learning the weight does not work at all. Before we fixed the annotation frame rate, the gap between the default and the best constant was much smaller, and the default looked nearly optimal. We list them because anyone who reuses VSI's scores or LongVideoBench's annotations can easily make the same mistakes.

8. Discussion

Lessons. For methods like VSI that mix two fixed models with one weight, the first step should be to tune that weight on the target data; this alone gives most of the gain. Any method that learns the weight should be compared with this tuned constant, not with the default from the original paper; otherwise the benefit of learning is overstated. When the only thing to learn is one number, different tasks do not compete much, forgetting stays small, and continual learning methods differ little. The harder problem is predicting per-question differences from very sparse supervision.

Limitations. We use one question per video, so each test split has only about 229 questions, which limits the statistical power of the QA experiment. With about two annotated moments per question, recall@8 is coarse. We use one dataset, one VLM, and a short sequence of four tasks.

Problem	What happened	Fix
Frames vs. seconds	The annotations are frame numbers, but the score curves are indexed by seconds. Comparing them directly gave zero recall for 94% of the questions.	Convert everything to seconds.
Annotation frame rate	LongVideoBench counts annotated frames at a fixed 24 fps, while the videos run at a median of 30 fps. Using the real frame rate moved the annotations too early. On 39 questions whose answer appears in a subtitle, 38 annotations fall within 5 s of that subtitle at 24 fps, but only 14 at the real frame rate (appendix, Fig. 8).	Always divide by 24.
Short videos dropped	VSI packs frames into a 4×4 grid; videos of 16 s or less cannot fill it and were silently skipped (197 questions, mostly text tasks).	Shrink the grid for short videos.
Training vs. testing the weight	Training used the probability-weighted average of λ, but testing used the single most likely value. The model put its probability on $\lambda = 0$ and $\lambda = 1$, so testing picked the two worst weights, and learned weights looked 3–5 points worse than the default (appendix, Fig. 9).	Use the weighted average at test time too.

Table 2. Problems found and fixed during the project.

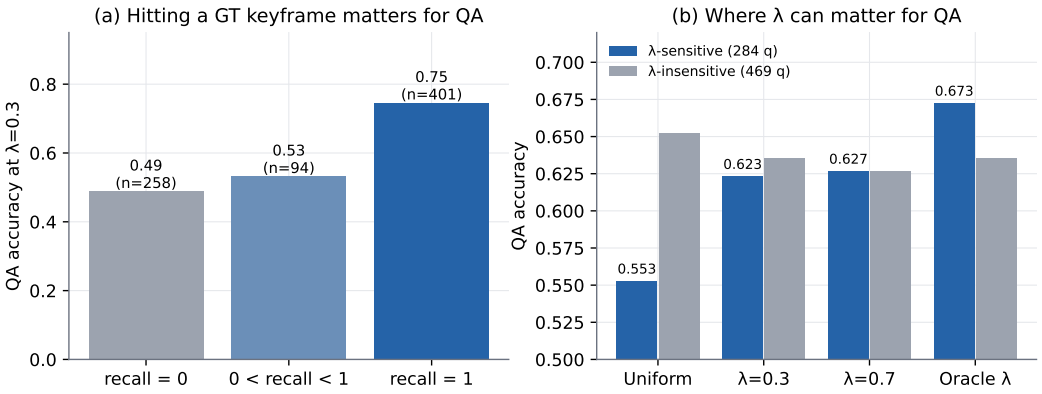


Figure 7. **The weight only matters for some questions.** (a) QA accuracy at $\lambda = 0.3$, grouped by how many annotated moments the 8 frames hit. (b) QA accuracy on the 284 questions where λ changes the selected frames and on the 469 where it does not.

Frames chosen by	QA accuracy	vs. $\lambda = 0.3$ (paired)
All 753 questions		
Even spacing	0.615	
$\lambda = 0$ (visual only)	0.604	
$\lambda = 0.3$ (default)	0.631	
$\lambda = 0.7$ (best recall)	0.627	
$\lambda = 1$ (subtitles only)	0.615	
Oracle λ	0.649	
Test questions of the predictors (3×229)		
$\lambda = 0.3$ (default)	0.648	–
LwF predictor	0.652	+0.4 [−1.5, +2.3]
Naive predictor	0.643	−0.4 [−2.3, +1.5]
Oracle λ	0.658	+1.0

Table 3. **Question answering with Qwen3-VL-8B.** Brackets: 95% bootstrap confidence interval of the paired difference. The LwF difference is not significant (McNemar $p = 0.77$).

9. Conclusion

We studied whether the weight that mixes the visual and subtitle scores in VSI should change with the question, and whether it can be learned as question types arrive one after another. The best weight clearly depends on the question type, but the continually trained predictor collapses to almost one weight for all types, so it only ties a well-tuned constant. Better frames improve answers only for the minority of questions where the weight changes the frames. Most of the remaining gain lies in per-question differences, and learning them from sparse supervision is the key problem for future work.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-VL Technical Report. arXiv:2511.21631, 2025. 4

[2] Arslan Chaudhry, Puneet K. Dokania, Thalaiyasingam Ajanthan, and Philip H. S. Torr. Riemannian Walk for Incremen-

tal Learning: Understanding Forgetting and Intransigence. In *European Conference on Computer Vision (ECCV)*, 2018. arXiv:1801.10112. 2

[3] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On Tiny Episodic Memories in Continual Learning. arXiv:1902.10486, 2019. 2, 3

[4] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-Time Open-Vocabulary Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2401.17270. 1

[5] Jianxiang He, Meisheng Hong, Jungang Li, Ziyang Chen, Weiyu Guo, Xuming Hu, and Hui Xiong. VSI: Visual Subtitle Integration for Keyframe Selection to enhance Long Video Understanding. arXiv:2508.06869, 2025. 1, 2

[6] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017. arXiv:1612.00796. 2

[7] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Ales Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021. arXiv:1909.08383. 2

[8] Zhizhong Li and Derek Hoiem. Learning without Forgetting. In *European Conference on Computer Vision (ECCV)*, 2016. arXiv:1606.09282. 2

[9] Shuming Liu, Chen Zhao, Tianqi Xu, and Bernard Ghanem. BOLT: Boost Large Vision-Language Model Without Training for Long-form Video Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2503.21483. 1, 2

[10] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient Episodic Memory for Continual Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.08840. 2

[11] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. arXiv:1908.10084. 1, 9

[12] Xi Tang, Jihao Qiu, Lingxi Xie, Yunjie Tian, Jianbin Jiao, and Qixiang Ye. Adaptive Keyframe Sampling for Long Video Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2502.21271. 1, 2

[13] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A Comprehensive Survey of Continual Learning: Theory, Method and Application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. arXiv:2302.00487. 2

[14] Haoning Wu, Dongxu Li, Bei Chen, and Junnan Li. LongVideoBench: A Benchmark for Long-context Interleaved Video-Language Understanding. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. arXiv:2407.15754. 2

[15] Zihui Xue and Radu Marculescu. Dynamic Multimodal Fusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023. arXiv:2204.00102. 2

[16] Jinhui Ye, Zihan Wang, Haosen Sun, Keshigeyan Chandrasegaran, Zane Durante, Cristobal Eyzaguirre, Yonatan Bisk, Juan Carlos Niebles, Ehsan Adeli, Li Fei-Fei, Jiajun Wu, and Manling Li. T*: Re-thinking Temporal Search for Long-Form Video Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2504.02259. 1, 2

Appendix

A. Details of the Two Fixes

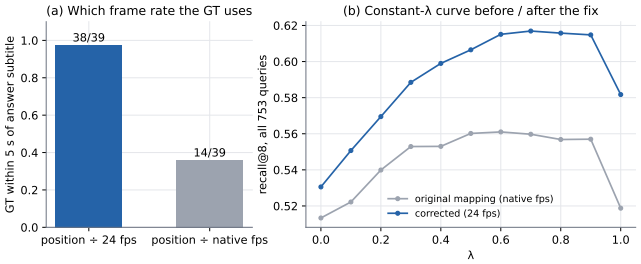


Figure 8. (a) How well the annotations line up with the answer subtitle under the two frame-rate assumptions. (b) Recall as a function of λ before and after the fix.

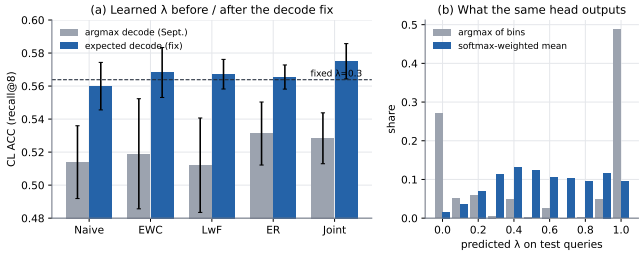


Figure 9. (a) Continual learning ACC before and after using the weighted average at test time (old annotation mapping). (b) The same predictor's output weights under the two choices.

B. Additional Results

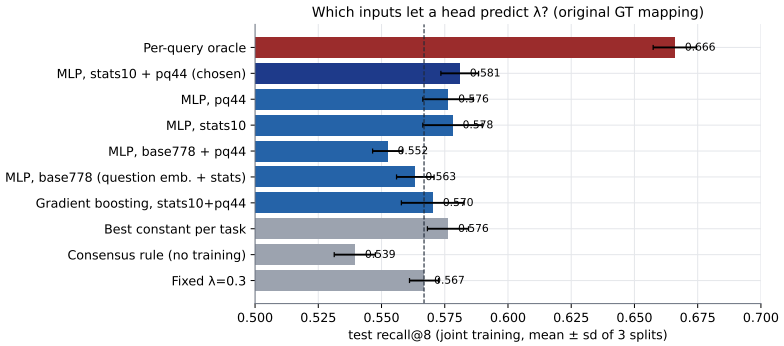


Figure 10. Test recall of a predictor trained on all tasks together with different inputs (old annotation mapping, mean $\pm$ std of 3 splits). The question-text embedding makes results worse; curve statistics plus our 44 per-question features work best.

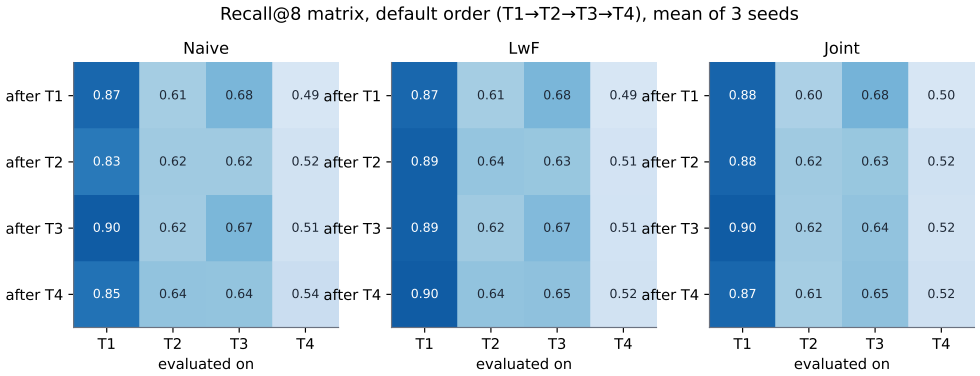


Figure 11. Recall matrix: row i is the test recall on each task after training on the i-th task (default order, mean of 3 seeds).

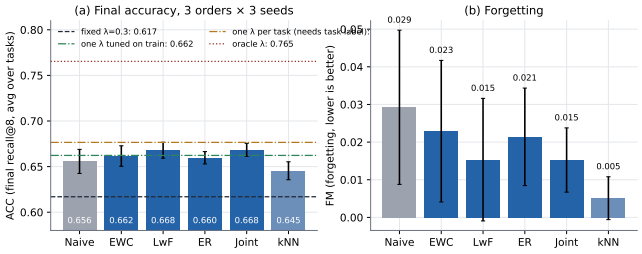


Figure 12. Bar-chart version of Tab. 1: (a) final accuracy with the baselines; (b) forgetting.

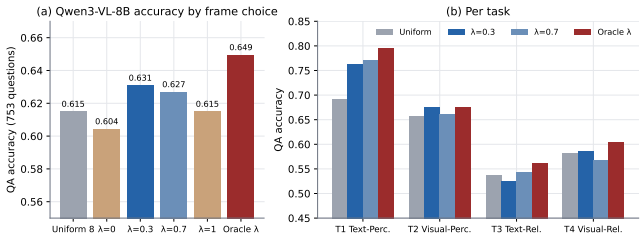


Figure 13. QA accuracy for all questions (a) and per task (b).

Method	ACC, basic inputs	ACC, + 44 per-question features	FM	vs. tuned constant
Naive	0.560	0.578	0.028	+0.5 (7/9)
EWC	0.568	0.587	0.017	+1.3 (9/9)
LwF	0.567	0.589	0.016	+1.5 (8/9)
ER	0.565	0.585	0.019	+1.1 (8/9)
Joint	0.575	0.589	0.018	+1.5 (9/9)
Default 0.3 / tuned constant / per-task constant / oracle: 0.564 / 0.574 / 0.584 / 0.673				

Table 4. Results with the old (wrong) annotation frame rate, kept as an ablation of the input features. Numbers are not comparable with Tab. 1. Here the per-question features gave a 1.5-point gain over the tuned constant; after the fix this gain shrank to 0.6, so we treat it with caution.

C. Implementation Details

- **Scores.** Official VSI code; YOLO-World-S detector; subtitle branch with the sentence embedding model all-mpnet-base-v2 [11]; one score per second; both scores rescaled to $[0, 1]$ with min-max scaling.
- **44 per-question features.** For each curve (8 each): share of zeros, whether the curve is constant, mean, standard deviation, share of the total score held by the top 8 seconds, how spread out in time the top 8 seconds are, log number of peaks, and the gap between the 8th and 9th highest score. Between curves (4): Pearson and Spearman correlation, and overlap of their top-8 and top-32 seconds within ± 5 s. Sensitivity to λ (12): overlap of the top 8 at each grid value with the top 8 at $\lambda = 0.3$, and the number of distinct top-8 sets. Agreement (12): for each grid value, how much its top 8 agrees with the others, plus the most agreed-on value.
- **Continual learning.** EWC weight 400; LwF temperature 2 and weight 1; ER buffer of 200 questions; kNN with $k = 5$ on standardized inputs; weight decay 10^{-4}.
- **Question answering.** Qwen3-VL-8B-Instruct in bf16, at most 448×448 pixels per frame, greedy decoding, at most 8 new tokens. For each frame we add the subtitles within ± 1 s of its time; for clipped videos the subtitle times are shifted by the clip start.